LAPORAN PRAKTIKUM PERTEMUAN 4

PEMROGRAMAN MOBILE CR002

Mengambil Data dari REST API (HTTP, Async, dan FutureBuilder)

Disusun oleh:

Septiara Pratama 20240801027

Dosen Pengampu:

JEFRY SUNUPURWA ASRI, S.Kom., M.Kom

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

TANGERANG

2026

Pelaksanaan Praktikum

Praktikum Pemrograman Mobile Pertemuan 4 membahas cara mengambil data dari REST API menggunakan Flutter. Pada praktikum ini, mahasiswa mempelajari pemrograman asinkron dengan Future, async, dan await, mengambil data menggunakan paket http, mengubah JSON menjadi objek model, serta menampilkan data menggunakan FutureBuilder.

API yang digunakan adalah JSONPlaceholder, yaitu API publik yang menyediakan data untuk keperluan latihan. Data pengguna ditampilkan dalam bentuk daftar, kemudian dapat dibuka untuk melihat informasi detail. Praktikum ini juga mempelajari cara menangani loading, kesalahan jaringan, dan menyediakan tombol untuk mencoba mengambil data kembali.

Tujuan Praktikum

- Memahami penggunaan Future, async, dan await pada Dart.
- Mengambil data dari REST API menggunakan paket http.
- Mengubah data JSON menjadi objek menggunakan model dan fromJson.
- Menampilkan data asinkron menggunakan FutureBuilder.
- Menangani proses loading, error, dan menyediakan tombol coba lagi.

Alat dan Bahan

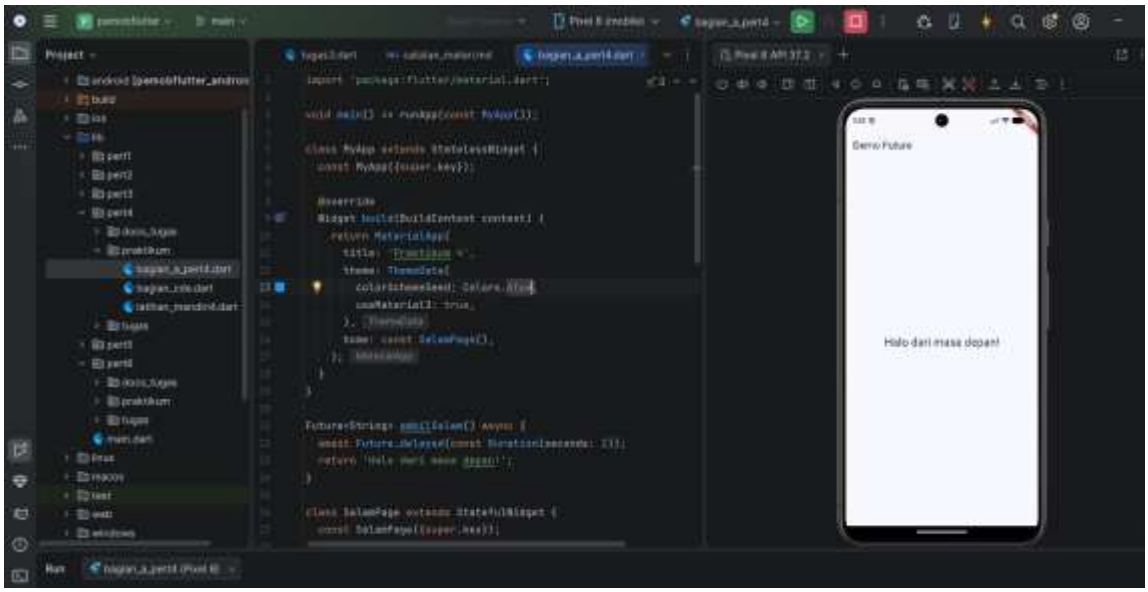
- Flutter SDK.
- Visual Studio Code atau editor lainnya.
- Emulator Android atau perangkat untuk menjalankan aplikasi.
- Koneksi internet.
- Paket Flutter http.
- API JSONPlaceholder dengan alamat https://jsonplaceholder.typicode.com/users.

Bagian A – Mengenal Future dan FutureBuilder

Pada bagian pertama, dibuat aplikasi sederhana untuk memahami cara kerja Future dan FutureBuilder tanpa menggunakan koneksi internet. Fungsi ambilSalam() digunakan untuk menunda pengembalian teks selama dua detik dengan Future.delayed().

Future disimpan dalam variabel _future melalui fungsi initState(). Selanjutnya, FutureBuilder digunakan untuk menampilkan indikator loading selama proses menunggu. Setelah dua detik, indikator tersebut berubah menjadi teks “Halo dari masa depan!”.

Hasil Praktikum:



Berdasarkan checkpoint modul, aplikasi menampilkan indikator loading selama dua detik, kemudian menampilkan teks salam. Hal ini menunjukkan bahwa proses asinkron dapat berjalan tanpa membuat tampilan aplikasi membeku.

Bagian B – Menyiapkan Paket HTTP

Pada bagian ini, paket http ditambahkan ke proyek Flutter melalui terminal menggunakan perintah berikut:

flutter pub add http

Paket http digunakan untuk mengirim permintaan ke REST API dan menerima respons dari server. Selain itu, izin internet perlu diperhatikan pada berkas AndroidManifest.xml, khususnya ketika aplikasi dibuat dalam mode release Android.

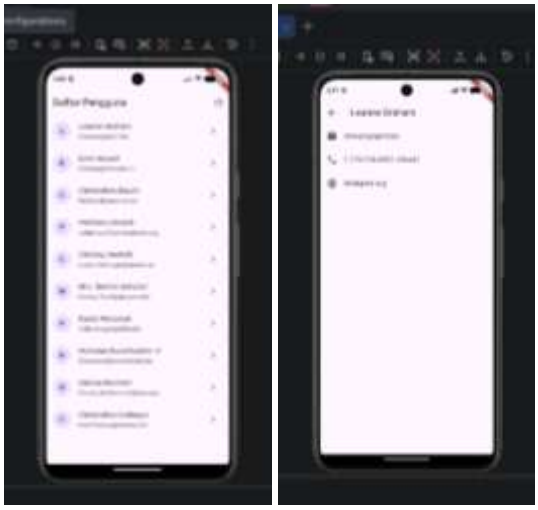
Izin yang digunakan adalah android.permission.INTERNET, yang ditempatkan sebelum tag <application> pada berkas manifest.

Hasil Praktikum:



Paket HTTP siap digunakan untuk mengambil data dari internet setelah proses instalasi dependensi selesai.

Bagian C - E



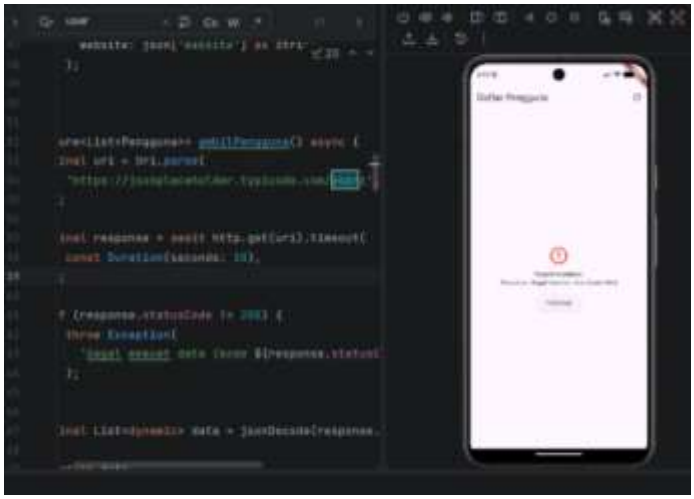
Bagian C, dibuat model Pengguna yang berisi data ID, nama, email, nomor telepon, dan website. Model ini digunakan untuk mengubah data JSON dari API menjadi objek Dart. Selain itu, dibuat fungsi ambilPengguna() yang mengambil data dari **https://jsonplaceholder.typicode.com/users** menggunakan package http. Jika respons API tidak memiliki status kode 200, fungsi akan menghasilkan Exception.

Pada Bagian D, data yang diperoleh dari API ditampilkan menggunakan FutureBuilder. Komponen ini menangani tiga kondisi, yaitu menampilkan indikator loading ketika data sedang diproses, menampilkan pesan kesalahan jika pengambilan data gagal, serta menampilkan daftar pengguna jika data berhasil diterima. **Pada Bagian E,** dilakukan pengujian untuk memastikan aplikasi dapat mengambil data dari API dan menampilkannya dengan benar. Dengan menggabungkan ketiga bagian tersebut, aplikasi Flutter dapat mengambil, mengolah, dan menampilkan data secara dinamis tanpa harus memasukkan data pengguna secara manual.

Bagian F – Percobaan Penanganan Galat

Pada bagian ini, dilakukan dua percobaan untuk memahami penanganan kesalahan ketika mengambil data dari REST API.

Percobaan pertama: URL tidak ditemukan



Alamat endpoint diubah menjadi /userz untuk menghasilkan respons 404. Karena kode respons bukan 200, fungsi pengambilan data melempar Exception. Kesalahan tersebut kemudian ditampilkan melalui snapshot.error, disertai tombol “Coba lagi”.

Setelah pengujian, URL dikembalikan ke alamat yang benar dan tombol coba lagi digunakan untuk memuat data kembali.

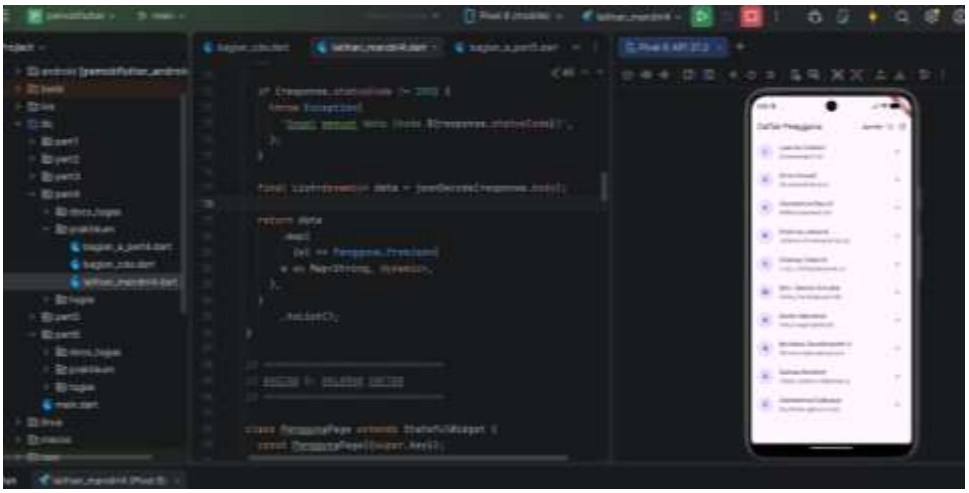
Percobaan kedua: koneksi internet dimatikan

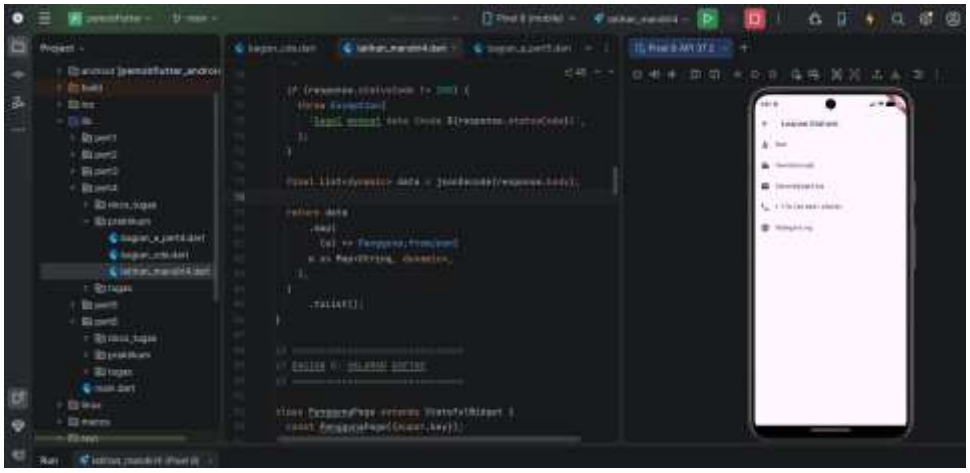


Koneksi internet perangkat atau emulator dimatikan, kemudian tombol refresh ditekan. Ketika permintaan tidak dapat mencapai server, aplikasi menampilkan pesan kesalahan jaringan. Setelah koneksi internet diaktifkan kembali, tombol coba lagi digunakan untuk mengulangi permintaan.

Hasil Praktikum: Kedua percobaan menunjukkan pentingnya penanganan error pada aplikasi yang menggunakan API. Aplikasi tidak hanya perlu menampilkan data ketika berhasil, tetapi juga perlu memberikan informasi ketika permintaan gagal.

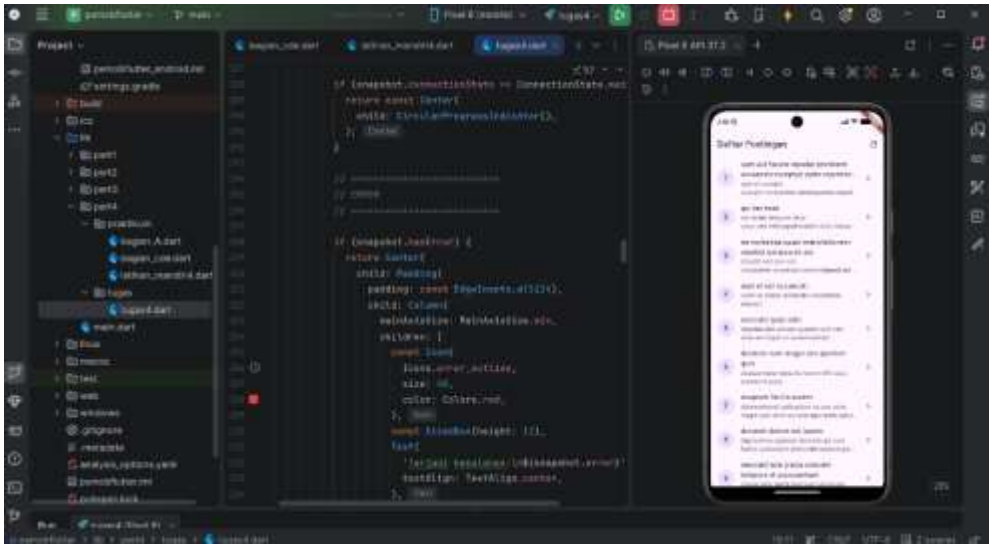
Latihan Mandiri

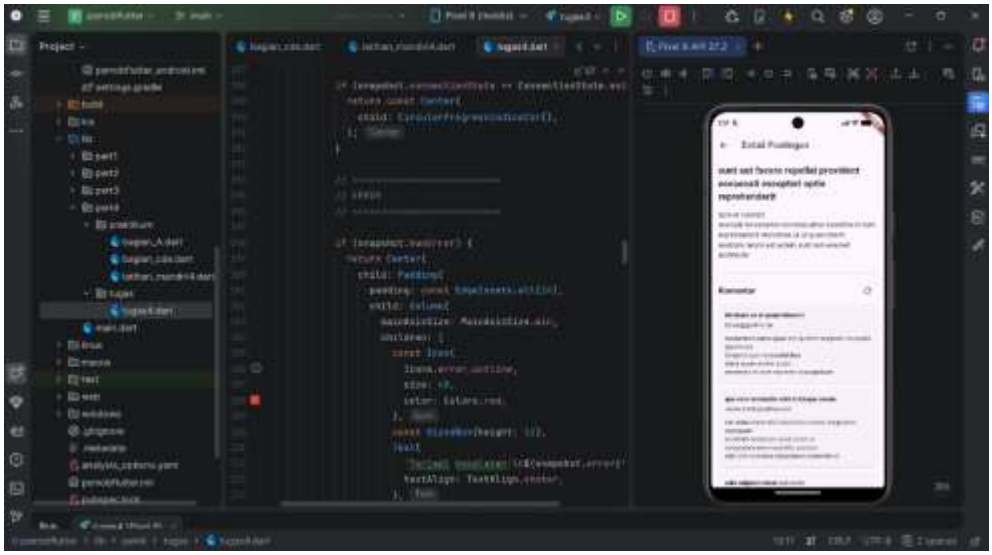


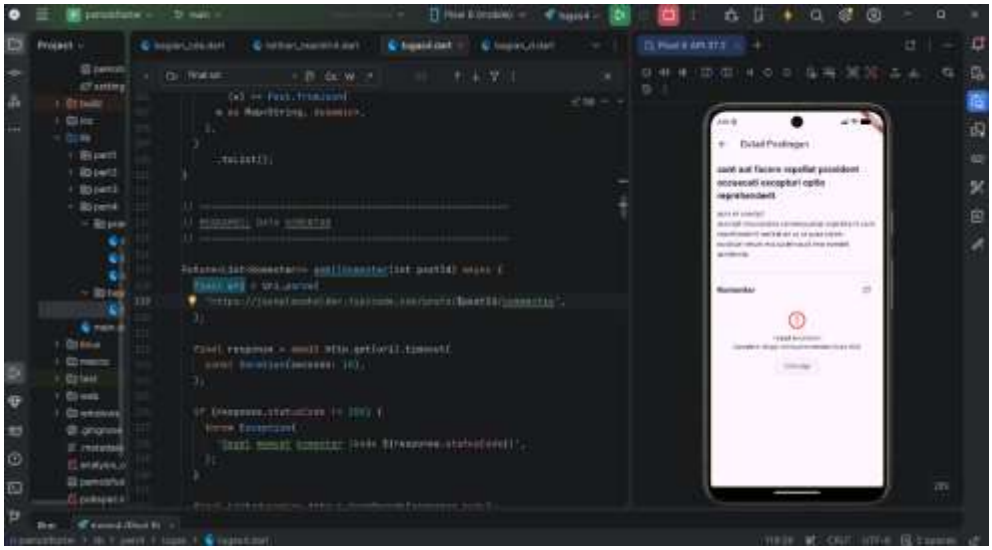


Pada latihan mandiri, dibuat aplikasi Flutter sederhana untuk mengambil dan menampilkan data postingan dari **REST API https://jsonplaceholder.typicode.com/posts.** Data yang diperoleh berupa ID, user ID, judul, dan isi postingan, kemudian diubah dari JSON menjadi objek Dart dan ditampilkan dalam bentuk daftar menggunakan FutureBuilder. Aplikasi juga menangani kondisi loading saat data sedang diambil dan menampilkan pesan kesalahan jika terjadi masalah dalam proses pengambilan data. Dari latihan ini, dapat dipahami cara menggunakan REST API pada Flutter, mengolah data JSON, serta menampilkan data secara dinamis pada aplikasi.

Tugas – Aplikasi Daftar Postingan







Pada bagian tugas, mahasiswa diminta membuat aplikasi Daftar Postingan dengan memanfaatkan REST API JSONPlaceholder. Aplikasi harus menampilkan daftar postingan yang berisi judul dan potongan isi postingan.

Ketika salah satu postingan dipilih, aplikasi membuka halaman detail yang menampilkan isi lengkap dan daftar komentar. Data postingan diambil melalui endpoint /posts, sedangkan komentar diambil melalui endpoint /posts/{id}/comments.

Aplikasi juga perlu menggunakan model Post dan Komentar dengan konstruktor fromJson. Kode pengambilan data dipisahkan dari kode antarmuka agar lebih mudah dipelihara.

Pada halaman daftar dan detail, aplikasi harus menangani kondisi loading dan error. Apabila terjadi kesalahan, pengguna harus dapat mencoba mengambil data kembali melalui tombol yang disediakan.

Hasil yang diharapkan: Aplikasi dapat menampilkan daftar postingan, membuka detail postingan beserta komentar, serta menangani kondisi loading dan kesalahan pengambilan data. Bukti pengerjaan berupa screenshot halaman daftar, detail, dan tampilan error serta berkas main.dart atau tautan repositori.

Jawaban Pertanyaan Refleksi

1. **Apa yang terjadi jika Future dibuat di dalam build() dan bukan di initState()? Mengapa?**
   Future dapat dibuat ulang setiap kali build() dijalankan sehingga permintaan data bisa berulang dan proses loading dapat terus terjadi. Karena itu, Future sebaiknya disimpan di initState() jika tidak perlu dibuat ulang setiap kali tampilan dibangun.
2. **Apa perbedaan snapshot.hasError dengan memeriksa response.statusCode? Mengapa keduanya diperlukan?**
   response.statusCode digunakan untuk memeriksa kode status HTTP dari server, misalnya 200 atau 404. Sementara itu, snapshot.hasError digunakan untuk memeriksa apakah proses Future menghasilkan kesalahan. Keduanya diperlukan karena kegagalan dapat berasal dari respons server maupun masalah jaringan atau proses lainnya.
3. **Mengapa data JSON sebaiknya diubah menjadi class model, bukan dipakai langsung sebagai Map?**
   Class model membuat data lebih terstruktur, memudahkan akses atribut, dan membantu memisahkan pengolahan data dari tampilan aplikasi. Dengan model, data dapat diakses melalui atribut seperti pengguna.name tanpa harus mengambil nilai dari Map berulang kali.
4. **Mengapa UI wajib menyediakan status loading dan error, bukan hanya status data?**
   Loading memberi tahu pengguna bahwa aplikasi sedang mengambil data, sedangkan error memberikan informasi ketika permintaan gagal. Dengan begitu, pengguna tidak hanya melihat layar kosong ketika proses berlangsung atau mengalami masalah.

Kesimpulan

Berdasarkan materi Praktikum Flutter Fundamental Pertemuan 4, pengambilan data dari REST API membutuhkan pemahaman mengenai proses asinkron, HTTP, JSON, model data, dan FutureBuilder. Penggunaan async dan await membantu mengelola proses pengambilan data, sedangkan model fromJson membuat data JSON lebih mudah diolah.

FutureBuilder memungkinkan aplikasi menampilkan kondisi loading, error, dan data sesuai dengan hasil proses yang berlangsung. Selain itu, penanganan kesalahan dan tombol coba lagi diperlukan agar aplikasi tetap mudah digunakan ketika terjadi masalah jaringan.

Melalui praktikum ini, mahasiswa mempelajari dasar pembuatan aplikasi Flutter yang mengambil data dari internet, menampilkan daftar pengguna, membuka halaman detail, serta menangani berbagai kondisi selama proses pengambilan data. Materi ini menjadi dasar untuk mengembangkan aplikasi yang menggunakan API dengan struktur kode yang lebih teratur dan antarmuka yang responsif.